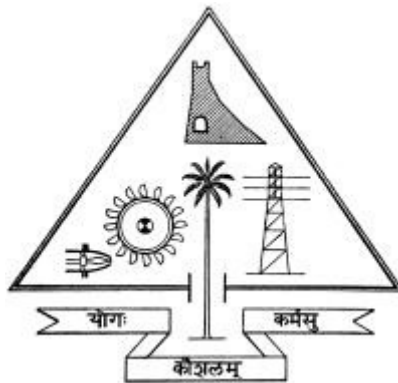


FLAWFIX SECUREASSIST: AN AI-POWERED VISUAL STUDIO CODE EXTENSION FOR REAL-TIME VULNERABILITY DETECTION AND INTELLIGENT SECURE PATCH GENERATION

*Main Project Report submitted in partial fulfillment of the requirements for the
award of the degree of **Master of Computer Applications** of the **APJ Abdul
Kalam Technological University***

submitted by

**SURJITH S
(TCR25MCA-2053)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

OCTOBER 2026

ABSTRACT

Ensuring software security throughout the software development life cycle has become increasingly important due to the growing complexity of applications and the rising number of cybersecurity threats. Developers require intelligent tools that can identify vulnerabilities early, provide meaningful explanations, and assist in applying secure fixes without disrupting the development workflow. FlawFix SecureAssist is proposed as an AI-powered Visual Studio Code extension that provides real-time vulnerability detection and intelligent secure patch generation.

The proposed system utilizes LLVM Intermediate Representation (LLVM IR) to perform language-independent semantic analysis and generates Intermediate Semantic Vulnerability Contexts (iSeVCs) for accurate vulnerability detection. Large Language Models (LLMs) are used to analyze vulnerabilities, provide Explainable Artificial Intelligence (XAI)-based explanations, estimate severity levels, recommend secure coding practices, and automatically generate secure patches. The system also verifies the generated patches, performs AI-based code optimization to improve software performance without affecting functionality, and generates comprehensive security reports. By integrating vulnerability detection, explanation, patch generation, verification, code optimization, and reporting into a single Visual Studio Code extension, FlawFix SecureAssist simplifies the software security process, improves code quality, and supports the development of secure, reliable, and efficient software applications.

CONTENTS

List of Abbreviations	1
1 INTRODUCTION	2
2 LITERATURE REVIEW	5
3 ENVIRONMENTAL STUDY	10
3.1 System Configuration	10
3.1.1 Hardware Requirements	10
3.1.2 Software Requirements	11
3.1.3 Software Requirements for Development	12
3.1.4 Software Requirements for Development	12
3.2 Software Specification	13
3.2.1 Python	13
3.2.2 TypeScript	14
3.2.3 Node.js	14
3.2.4 Visual Studio Code	14
3.2.5 LLVM Framework	15
3.2.6 Large Language Model (LLM) Framework	15
3.2.7 Explainable Artificial Intelligence (XAI)	15
3.2.8 ReportLab	16
4 SYSTEM ANALYSIS	17
4.1 Requirements Analysis	17
4.2 Existing System	18
4.3 Limitations of Existing System	19
4.4 Proposed System	20
4.5 Advantages of Proposed System	21
4.6 Functional Requirements	22
4.6.1 VS Code Extension Installation and User Interaction . .	22
4.6.2 Source Code Monitoring and Input Handling	23
4.6.3 Language Processing and LLVM IR Generation	23
4.6.4 Semantic Vulnerability Context Extraction	24
4.6.5 AI-Based Vulnerability Detection Using LLM	25
4.6.6 Real-Time Vulnerability Visualization	26
4.6.7 Intelligent Secure Patch Generation	26
4.6.8 Security Report Generation	27
4.7 Non-Functional Requirements	27
4.7.1 Performance	27
4.7.2 Usability	28
4.7.3 Reliability	28
4.7.4 Maintainability	28
4.7.5 Explainability	28

4.8	Feasibility Study	29
4.8.1	Technical Feasibility	29
4.8.2	Operational Feasibility	29
4.8.3	Economical Feasibility	30
5	SYSTEM DESIGN	31
5.1	System Architecture	31
5.2	Data Flow Diagram (DFD)	32
5.2.1	DFD Level 0	33
5.2.2	DFD Level 1	33
5.3	List of Modules	35
5.3.1	FlawFix SecureAssist Extension	35
5.3.2	LLVM IR Generator	35
5.3.3	iSeVC Feature Extractor	35
5.3.4	LLM-Based Vulnerability Detection	36
5.3.5	Explainable AI	36
5.3.6	Secure Patch Generation	36
5.3.7	Patch Preview and Validation	36
5.3.8	Apply Patch	36
	Bibliography	38

LIST OF ABBREVIATIONS

AI	–	Artificial Intelligence
API	–	Application Programming Interface
CFG	–	Control Flow Graph
CWE	–	Common Weakness Enumeration
IDE	–	Integrated Development Environment
IR	–	Intermediate Representation
ISeVC	–	Intermediate Semantic Vulnerability Context
LLM	–	Large Language Model
LLVM IR	–	Low Level Virtual Machine Intermediate Representation
VS Code	–	Visual Studio Code
XAI	–	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

Software vulnerabilities continue to be one of the major security challenges in modern software development. Although numerous static analysis tools are available, they often produce false positives, provide limited explanations, and require developers to manually analyze, repair, and verify vulnerable code. This process is time-consuming and demands significant security expertise. Existing AI-based coding assistants mainly focus on code generation and completion rather than comprehensive vulnerability analysis, secure patch generation, and automated verification.

To address these limitations, this project proposes FlawFix SecureAssist, an AI-powered Visual Studio Code (VS Code) extension designed to provide real-time vulnerability detection, explainable security analysis, intelligent secure patch generation, and automated patch verification within the software development environment. The extension continuously analyzes source code during software development, enabling developers to receive timely security feedback without interrupting the development workflow.

The proposed system utilizes LLVM Intermediate Representation (LLVM IR) to perform language-independent semantic analysis of source code. Before LLVM IR generation, the system validates the source code to ensure that it is syntactically correct and can be successfully compiled into LLVM IR. If syntax or compilation errors are detected, the corresponding errors are reported to the developer, and vulnerability analysis is postponed until successful LLVM IR generation. By converting source code into LLVM IR, the system analyzes the actual behavior and semantics of the program instead of relying solely on syntax or predefined rules. Since software

vulnerabilities often exist in syntactically correct and compilable programs, the generated LLVM IR enables the system to analyze vulnerable yet compilable code. This approach enables accurate vulnerability detection across multiple programming languages while providing a common intermediate representation for security analysis.

To improve detection accuracy, the system generates Intermediate Semantic Vulnerability Contexts (iSeVCs) from the LLVM IR representation. iSeVCs capture rich semantic information by combining program instructions with their associated control-flow and data-flow contexts, creating meaningful vulnerability-oriented representations of the source code. These semantic contexts preserve the behavioral characteristics of the program, enabling the system to identify complex vulnerabilities that are difficult to detect using conventional syntax-based analysis. The generated iSeVCs are then provided to the AI-based Vulnerability Detection Engine, allowing the model to perform more accurate and context-aware security analysis.

Once a vulnerability is detected, the Vulnerability Detection Engine identifies the affected code region and forwards its semantic context to the Large Language Model (LLM). Rather than simply reporting an error, the LLM analyzes the vulnerability, determines its root cause, estimates its severity, explains the potential security impact, identifies the corresponding Common Weakness Enumeration (CWE) classification, and provides clear recommendations for secure coding practices. This enables developers to understand not only where the vulnerability exists but also why it occurs and how similar vulnerabilities can be prevented.

The Secure Patch Generator then produces AI-generated secure code that preserves the original functionality while eliminating the detected vulnerability. The system regenerates a secure version of the entire affected function by utilizing its complete semantic context. In addition to fixing security flaws, the generated patch may also improve code quality and optimize the implementation without altering the intended behavior of the application. If multiple vulnerable functions are detected, the system

independently generates secure function-level patches for each affected function. Furthermore, when appropriate, the system may generate multiple secure patch alternatives for a detected vulnerability, allowing developers to compare different secure solutions and select the most suitable one. The generated patch is displayed through an interactive Patch Preview interface that presents the original and modified code side by side, allowing developers to review the proposed changes before applying them.

After the developer approves a generated patch, the extension automatically updates the corresponding function within the source code and initiates the Patch Verification phase. During this stage, the modified program is re-analyzed using the LLVM IR analysis pipeline and regenerated iSeVC-based semantic contexts to verify that the vulnerability has been completely removed without introducing additional security issues. If vulnerabilities still remain, the system generates an improved function-level patch and repeats the verification process until the required security conditions are satisfied.

Finally, FlawFix SecureAssist generates a comprehensive Security Report containing detected vulnerabilities, affected functions, CWE classifications, severity levels, root-cause explanations, AI-generated secure function patches, verification results, secure coding recommendations, and code optimization suggestions. By integrating LLVM IR-based semantic analysis, iSeVC generation, Explainable Artificial Intelligence (XAI), intelligent function-level secure patch generation, AI-assisted code optimization, automated patch verification, and comprehensive security reporting into a single Visual Studio Code extension, FlawFix SecureAssist provides a complete Detect → Analyze → Explain → Patch → Verify workflow. The proposed system reduces manual effort, accelerates vulnerability remediation, improves developer productivity, enhances software quality, and promotes secure software development by providing intelligent, context-aware security assistance directly within the integrated development environment.

CHAPTER 2

LITERATURE REVIEW

Software security has become a critical area of research due to the increasing number of cyberattacks targeting software vulnerabilities. As software systems become larger and more complex, identifying and mitigating vulnerabilities during software development has become essential. Traditional static analysis tools rely on predefined rules and heuristics to detect insecure coding practices. Although these tools can identify common vulnerabilities, they often produce false positives and false negatives because they primarily analyze source-code syntax rather than understanding the actual semantics of program execution. Furthermore, most traditional tools are language-dependent and require developers to manually analyze vulnerabilities, implement secure fixes, and verify the correctness of the applied patches. This manual process is time-consuming, error-prone, and requires significant security expertise.

Recent research has focused on improving vulnerability detection using compiler technologies and Artificial Intelligence. Compiler-based approaches utilize LLVM Intermediate Representation (LLVM IR) to provide a language-independent representation of source code. Instead of analyzing syntax alone, LLVM IR captures the semantic behavior of programs, enabling vulnerability detection across multiple programming languages using a unified framework. Such semantic analysis has demonstrated improved detection accuracy by considering instruction behavior, control flow, and data flow rather than relying solely on predefined vulnerability patterns. However, most existing studies primarily concentrate on vulnerability detection and classification,

providing limited support for automated vulnerability repair or integration within modern development environments [1].

The emergence of Large Language Models (LLMs) has significantly advanced software vulnerability analysis. LLMs are capable of understanding source code, reasoning about program behavior, identifying security vulnerabilities, explaining their causes, and recommending secure coding practices. Compared with conventional rule-based techniques, AI-based approaches provide better contextual understanding and can identify previously unseen vulnerabilities [2]. Despite these advantages, many existing LLM-based solutions operate as standalone tools, requiring developers to manually submit source code for analysis. This interrupts the software development process and reduces their practical usability. More recent studies have explored integrating AI-assisted vulnerability detection directly into Integrated Development Environments (IDEs), allowing developers to receive real-time security feedback while writing code. Although these approaches improve developer productivity, they generally stop at vulnerability detection and recommendations, leaving developers responsible for manually implementing and validating security fixes [3].

Another important research direction is AI-assisted automated vulnerability repair. Recent studies have demonstrated that LLMs can generate secure patches that eliminate vulnerabilities while preserving program functionality [4]. However, AI-generated patches are not always reliable and often require manual review and testing before deployment. Many existing repair systems also lack comprehensive verification mechanisms to ensure that generated patches completely remove vulnerabilities without introducing new defects. Similarly, Explainable Artificial Intelligence (XAI) has gained attention in software security by providing detailed explanations of vulnerabilities, including root causes, severity, potential impacts, and mitigation strategies. Explainable AI helps developers understand vulnerabilities more effectively and promotes secure

coding practices, but many existing systems still provide only limited explanations [5].

Modern research has also emphasized the importance of Intermediate Semantic Vulnerability Contexts (iSeVCs) for improving vulnerability detection. Instead of analyzing isolated source-code statements, iSeVCs are generated from LLVM Intermediate Representation (LLVM IR) and capture semantic relationships among program instructions together with their surrounding control-flow and data-flow contexts. These semantic contexts provide richer representations of program behavior, enabling AI models to detect complex vulnerabilities more accurately across multiple programming languages [1]. Despite these advancements, existing research often investigates vulnerability detection, explainable AI, automated repair, and patch verification independently. As a result, developers frequently rely on multiple disconnected tools to complete the vulnerability remediation process, increasing development complexity and reducing productivity.

The proposed FlawFix SecureAssist addresses these limitations by integrating multiple software security capabilities into a single Visual Studio Code extension. Unlike conventional vulnerability scanners, the proposed system provides a complete Detect → Analyze → Explain → Patch → Verify workflow within the development environment. The extension continuously monitors source code during development, enabling real-time vulnerability detection without interrupting the coding process.

A key feature of the proposed system is its use of LLVM Intermediate Representation (LLVM IR) for language-independent semantic analysis. The generated LLVM IR is used to create Intermediate Semantic Vulnerability Contexts (iSeVCs), which preserve the semantic behavior of programs by combining relevant instructions with their surrounding control-flow and data-flow information. These semantic contexts provide meaningful inputs to the AI-based Vulnerability Detection Engine, significantly improving detection accuracy while supporting multiple programming languages.

After identifying vulnerabilities, the proposed system employs

Explainable Artificial Intelligence to analyze each vulnerability, determine its root cause, estimate its severity, explain potential security risks, and recommend secure coding practices. Rather than simply displaying warning messages, the system helps developers understand why vulnerabilities occur and how they can be prevented. The AI-powered Secure Patch Generator then automatically generates secure patches that preserve program functionality while eliminating vulnerabilities. The generated patches are presented through a side-by-side comparison interface, allowing developers to review proposed modifications before applying them.

Unlike many existing repair systems, FlawFix SecureAssist also performs Patch Verification. After a patch is applied, the modified source code is re-analyzed using LLVM IR and regenerated iSeVCs to ensure that vulnerabilities have been completely removed without introducing new security issues. This verification stage improves the reliability of automated vulnerability repair and increases developer confidence in AI-generated solutions.

In addition to vulnerability remediation, the proposed system incorporates AI-powered Code Optimization. While existing automated repair systems primarily focus on eliminating security flaws, they often ignore code efficiency and maintainability. FlawFix SecureAssist analyzes the repaired code to identify opportunities for improving computational efficiency without changing program functionality. The AI can recommend optimized algorithms, eliminate redundant computations, simplify control-flow structures, and improve memory utilization while preserving software security. These optimizations are presented alongside the generated secure patches, enabling developers to review both security and performance improvements before applying changes.

Finally, the system generates comprehensive Security Reports containing detected vulnerabilities, severity levels, CWE classifications, AI-generated explanations, secure patches, verification results, and secure coding recommendations. These reports support software maintenance, auditing, and future software improvements while providing developers with a clear

understanding of the application's security posture.

Overall, current research has made significant progress in AI-assisted vulnerability detection, semantic code analysis, explainable AI, and automated vulnerability repair [1]. However, most existing approaches focus on individual components rather than providing a unified solution. FlawFix SecureAssist bridges this research gap by combining LLVM IR-based semantic analysis, Intermediate Semantic Vulnerability Context (iSeVC) generation, Explainable AI, intelligent secure patch generation, AI-assisted code optimization, automated patch verification, comprehensive security reporting, and seamless Visual Studio Code integration into a single platform. By integrating these capabilities into one intelligent development environment, the proposed system reduces developer effort, improves remediation accuracy, enhances code quality, accelerates secure software development, and promotes the development of reliable and secure software applications.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter outlines the hardware and software requirements essential for developing and running the system. It details the necessary tools, platforms, and specifications to ensure smooth implementation and deployment within a compatible computing environment.

3.1 System Configuration

System configuration describes the hardware and software resources required to develop, test, and run the system efficiently. It includes the necessary computing environment, operating system, development tools, and software platforms required for implementing AI-based vulnerability detection and secure patch generation.

3.1.1 Hardware Requirements

Hardware requirements specify the physical components needed for developing, running, and testing the system. It includes the processor, memory, storage, and other computing resources required to ensure efficient system performance.

The key hardware requirements are detailed as follows:

- **Processor:** Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to efficiently handle code processing tasks.
- **Memory:** 8 GB (16 GB recommended for AI processing)
- **Storage:** Minimum 100 GB free disk space.

- **Connectivity:** Stable internet connection (essential for communicating with API-hosted Large Language Model endpoints)

3.1.2 Software Requirements

Software requirements define the software tools, platforms, frameworks, and technologies required for developing, testing, and executing the system. These requirements specify the necessary operating systems, programming environments, libraries, and supporting software needed for efficient system implementation.

The key software development requirements are detailed as follows:

- **Operating System:** Windows 10/11, Linux, or macOS.
- **Visual Studio Code:** Development environment and extension platform.
- **Programming Languages:**
 - Python(Backend Processing).
 - TypeScript (VS Code Extension).
 - Node.js(Extension Runtime).
- **Version Control System (Git):** Source code management and version control.
- **LLVM Framework:** Clang, rustc, GraalVM, and Nuitka for LLVM IR generation and semantic analysis.
- **LLM API:** Gemini API for vulnerability detection, explanation, secure patch generation, and code optimization.
- **Explainable AI (XAI):** Gemini API with structured prompting for root cause analysis and secure coding recommendations.

3.1.3 Software Requirements for Development

Software requirements for development specify the programming languages, development tools, frameworks, libraries, and platforms required to design, implement, test, and maintain the FlawFix SecureAssist system during the software development process.

3.1.4 Software Requirements for Development

Software requirements for development specify the programming languages, development tools, frameworks, libraries, and platforms required to design, implement, test, and maintain the FlawFix SecureAssist system during the software development process.

The key software development requirements are detailed as follows:

- **Operating System:** Windows 10/11, Linux, or macOS.
- **Development Platform:** Visual Studio Code.
- **Version Control System:** Git, for source code management and version control.
- **Visual Studio Code Extension API:** Used for developing the VS Code extension.
- **Programming Languages:**
 - Python 3.12 or later.
 - TypeScript 5.6 or later.
 - Node.js 20.x or later.
- **Python Libraries:**
 - Google Generative AI SDK: Used for integrating the Gemini API.
 - ReportLab: Used for generating comprehensive security reports in PDF format.

- **LLVM Framework:**
 - Clang: Used for generating LLVM IR from C/C++ source code.
 - rustc: Used for generating LLVM IR from Rust source code.
 - GraalVM: Used for Java compilation and LLVM-based analysis.
 - Nuitka: Used for compiling Python code for LLVM-based analysis.
- **LLM Framework:** Gemini API, used for vulnerability detection, explanation, secure patch generation, patch verification, and code optimization.

3.2 Software Specification

Software specification defines the functional and technical details of the software system, including the technologies, programming languages, frameworks, tools, and components required to design, develop, and implement the system. It describes how different software modules work together to achieve the system objectives.

3.2.1 Python

Python is a high-level, interpreted, and object-oriented programming language known for its simplicity, readability, and extensive collection of libraries. It is widely used in artificial intelligence, machine learning, automation, web development, and backend application development. Python provides powerful libraries for data processing, API integration, file handling, and automation, making it suitable for implementing intelligent software systems and backend services. In FlawFix SecureAssist, Python is used as the primary backend programming language for processing LLVM Intermediate Representation (LLVM IR), generating Intermediate Semantic Vulnerability Contexts (iSeVCs), integrating the Gemini API, performing vulnerability detection, generating secure patches, verifying patches, optimizing source code, and generating security reports.

3.2.2 TypeScript

TypeScript is an open-source, statically typed programming language developed by Microsoft that extends JavaScript with additional features such as static type checking and object-oriented programming support. It improves code reliability, maintainability, and scalability, making it well suited for developing large-scale applications, web applications, and Visual Studio Code extensions. In FlawFix SecureAssist, TypeScript is used to develop the Visual Studio Code extension, monitor source code changes, communicate with the Python backend, and display vulnerability analysis results, secure patch suggestions, and security reports within the editor.

3.2.3 Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that enables JavaScript and TypeScript applications to run outside a web browser. It provides an event-driven, non-blocking architecture suitable for developing scalable applications. Node.js is widely used for server-side development, application runtime, API communication, and executing Visual Studio Code extensions. In FlawFix SecureAssist, Node.js provides the runtime environment for the Visual Studio Code extension and enables communication between the extension and the backend processing modules.

3.2.4 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight, open-source source code editor developed by Microsoft. It supports multiple programming languages and provides features such as syntax highlighting, intelligent code completion, debugging, integrated terminal, version control integration, and extension support. Its extensible architecture makes it suitable for developing customized tools and software extensions. In FlawFix SecureAssist, Visual Studio Code serves as the primary development environment where developers receive real-time vulnerability detection, AI-generated explanations, secure patch suggestions, verified patches, optimized code,

and security reports directly while coding.

3.2.5 LLVM Framework

LLVM (Low Level Virtual Machine) is a modular compiler framework designed for program compilation, optimization, and static program analysis. It provides LLVM Intermediate Representation (LLVM IR), which serves as a language-independent representation of source code while preserving program semantics. LLVM supports multiple programming languages through different compiler frontends and enables advanced code analysis, optimization, and transformation. In FlawFix SecureAssist, Clang, rustc, GraalVM, and Nuitka are used to generate LLVM IR from supported programming languages. The generated LLVM IR is analyzed to extract semantic information and generate Intermediate Semantic Vulnerability Contexts (iSeVCs) for language-independent vulnerability detection.

3.2.6 Large Language Model (LLM) Framework

A Large Language Model (LLM) framework provides access to advanced artificial intelligence models capable of understanding, analyzing, and generating natural language and source code. It enables tasks such as code understanding, reasoning, text generation, vulnerability analysis, automated code generation, and intelligent programming assistance. LLM frameworks improve software development by providing context-aware recommendations and automated assistance. In FlawFix SecureAssist, the Gemini API is used as the LLM framework to analyze semantic vulnerability contexts, detect vulnerabilities, explain root causes, classify severity, generate secure patch suggestions, verify generated patches, and optimize source code.

3.2.7 Explainable Artificial Intelligence (XAI)

Explainable Artificial Intelligence (XAI) is an approach that improves the transparency and interpretability of artificial intelligence systems by

providing understandable explanations for their decisions and predictions. It helps users understand how AI-generated results are produced, increasing trust, accountability, and decision-making confidence in intelligent software systems. In FlawFix SecureAssist, XAI is used to provide root-cause analysis, severity explanations, security impact, and secure coding recommendations, enabling developers to understand detected vulnerabilities and apply secure fixes with confidence.

3.2.8 ReportLab

ReportLab is an open-source Python library used for generating PDF documents programmatically. It provides features for creating professionally formatted reports containing text, tables, images, charts, and graphical elements. It is widely used for generating automated reports, invoices, certificates, and other structured documents in PDF format. In FlawFix SecureAssist, ReportLab is used to generate comprehensive security reports containing vulnerability summaries, vulnerability types, severity levels, root-cause analysis, secure patch suggestions, patch verification results, code optimization details, and overall security assessment in PDF format.

CHAPTER 4

SYSTEM ANALYSIS

System analysis is the process of studying the requirements, objectives, and functionality of a proposed system to develop an effective solution. It involves identifying user needs, analyzing system components, and ensuring that all modules work together efficiently to achieve the desired objectives.

For the proposed system, system analysis focuses on overcoming the limitations of traditional vulnerability detection tools by integrating LLVM Intermediate Representation (LLVM IR), Intermediate Semantic Vulnerability Contexts (iSeVCs), Large Language Models (LLMs), and Explainable Artificial Intelligence (XAI) within the Visual Studio Code environment. The system provides real-time vulnerability detection, AI-based explanations, secure patch generation, patch verification, and comprehensive security reporting. This analysis ensures that the proposed system is accurate, reliable, scalable, and capable of supporting secure software development.

4.1 Requirements Analysis

Requirement analysis is a crucial phase of the Software Development Life Cycle (SDLC) that identifies the functional and non-functional requirements of the proposed system. For FlawFix SecureAssist, it focuses on developing an AI-powered Visual Studio Code extension for real-time vulnerability detection, analysis, and secure patch generation.

The proposed system uses LLVM Intermediate Representation (LLVM IR) to perform language-independent semantic analysis and generate Intermediate Semantic Vulnerability Contexts (iSeVCs) for accurate vulnerability detection. Large Language Models (LLMs) are employed to

analyze vulnerabilities, explain their root causes and severity, generate secure patches, and recommend secure coding practices using Explainable AI (XAI). The system also verifies the generated patches to ensure vulnerabilities are completely removed and incorporates an AI-powered Code Optimization module to improve code efficiency without affecting functionality. Finally, it generates comprehensive security reports, providing developers with an integrated solution for secure software development within the Visual Studio Code environment.

4.2 Existing System

The existing system for software vulnerability detection primarily relies on traditional static analysis tools and AI-based standalone vulnerability detection systems. Traditional tools analyze source code using predefined rules, signatures, and pattern-matching techniques to identify common security vulnerabilities. Although these tools are effective in detecting known vulnerabilities, they often produce false positives and false negatives because they focus mainly on source-code syntax rather than understanding the semantic behavior of programs.

Recent AI-based approaches employ Large Language Models (LLMs) to improve vulnerability detection and provide security recommendations. Some systems, such as FlawFix, utilize LLVM Intermediate Representation (LLVM IR) and Intermediate Semantic Vulnerability Contexts (iSeVCs) to perform language-independent semantic analysis and improve detection accuracy. However, these systems primarily focus on vulnerability detection and explanation, while secure patch generation, patch verification, and code optimization remain limited or require manual intervention.

As a result, developers often need to use multiple disconnected tools to detect vulnerabilities, understand security issues, generate fixes, verify patches, and optimize code, making the software development process more complex and time-consuming.

4.3 Limitations of Existing System

- **Rule-Based Vulnerability Detection:**

- Most existing vulnerability detection tools rely on predefined rules, signatures, and pattern-matching techniques to identify security flaws. While these methods are effective for known vulnerabilities, they cannot accurately detect complex, evolving, or previously unseen vulnerabilities. As software becomes more sophisticated, rule-based approaches become less effective and require frequent updates.

- **High False Positives and False Negatives:**

- Traditional static analysis tools often generate false positives by reporting secure code as vulnerable and false negatives by failing to detect actual security issues. These inaccurate results increase the time required for manual verification and reduce developers' confidence in the analysis results.

- **Limited Vulnerability Explanation:**

- Most existing tools simply display warning messages or vulnerability names without providing sufficient information about the root cause, severity, potential impact, or recommended mitigation strategies. This makes it difficult for developers, especially those with limited security knowledge, to understand and resolve vulnerabilities effectively.

- **Poor Development Environment Integration:**

- Many AI-based vulnerability detection systems operate as standalone applications or external web services. Developers must leave their coding environment, submit source code for analysis, and review results separately. This interrupts the software development workflow and reduces developer productivity.

- **Lack of Code Optimization:**

- Most existing vulnerability detection and repair systems focus only on identifying and fixing security vulnerabilities. They do not analyze the repaired code for performance improvements, algorithm optimization, memory efficiency, or code maintainability, resulting in secure but not necessarily optimized code.

- **Limited Semantic Analysis:**

- Many existing systems focus mainly on the syntax and structure of source code rather than understanding its semantic behavior. They often ignore important information such as control flow, data flow, and program execution logic, which limits their ability to identify complex vulnerabilities accurately.

4.4 Proposed System

The proposed system, **FlawFix SecureAssist: An AI-Powered Visual Studio Code Extension for Real-Time Vulnerability Detection and Intelligent Secure Patch Generation**, provides an integrated solution for detecting and mitigating software vulnerabilities during software development. The system combines LLVM Intermediate Representation (LLVM IR), Intermediate Semantic Vulnerability Contexts (iSeVCs), Large Language Models (LLMs), Explainable Artificial Intelligence (XAI), secure patch generation, patch verification, AI-based code optimization, and security report generation within the Visual Studio Code environment. It performs real-time semantic vulnerability analysis on syntactically correct and compilable functions across multiple programming languages, including C, C++, Rust, Java, and Python.

The system converts source code into LLVM Intermediate Representation (LLVM IR) using language-specific compiler frameworks, extracts semantic vulnerability contexts, and utilizes Large Language Models (LLMs) to detect vulnerabilities, classify their severity, and identify vulnerable functions.

Explainable Artificial Intelligence (XAI) provides root-cause analysis, vulnerability explanations, and secure coding recommendations to help developers understand and remediate security issues effectively.

For each detected vulnerability, the system automatically generates one or more secure patch suggestions, verifies the generated patches to ensure vulnerabilities are eliminated while preserving the original functionality, and performs AI-based code optimization to improve code quality and efficiency. Finally, the system generates a comprehensive security report containing vulnerability details, severity levels, root-cause analysis, secure coding recommendations, patch suggestions, verification results, and code optimization summaries. By integrating all these capabilities into a single Visual Studio Code extension, the proposed system simplifies vulnerability remediation, enhances developer productivity, and promotes secure software development.

4.5 Advantages of Proposed System

- **Real-Time Vulnerability Detection:**

- The system continuously monitors source code and detects vulnerabilities while developers are writing code, enabling early identification of security issues

- **Language-Independent Analysis:**

- By using LLVM Intermediate Representation (LLVM IR), the system performs semantic analysis across multiple programming languages through a unified framework..

- **Accurate Vulnerability Detection:**

- The use of Intermediate Semantic Vulnerability Contexts (iSeVCs) and Large Language Models (LLMs) improves detection accuracy by analyzing the semantic behavior of programs instead of relying only on syntax.

- **Explainable AI Support:**

- The system provides detailed explanations of detected vulnerabilities, including their root cause, severity, potential impact, and secure coding recommendations.

- **Intelligent Secure Patch Generation:**

- The proposed system automatically generates secure patches that eliminate vulnerabilities while preserving the original functionality of the application.

- **AI-Based Code Optimization:**

- The system recommends performance improvements by optimizing algorithms, reducing redundant computations, and improving code efficiency without affecting functionality.

- **Automated Patch Verification:**

- The system recommends performance improvements by optimizing algorithms, reducing redundant computations, and improving code efficiency without affecting functionality.

4.6 Functional Requirements

This section describes the core functionalities that the system must provide to meet user requirements. Functional requirements define the specific operations and behaviors of the system when analyzing source code and generating documentation. They outline what the system should do and serve as a foundation for designing and developing its various modules.

4.6.1 VS Code Extension Installation and User Interaction

- The system shall be provided as a Visual Studio Code extension that can be installed and activated within the VS Code environment.

- The extension shall integrate with the VS Code editor interface and provide security analysis features directly within the development workflow.
- The system shall allow users to Open or write source code files, Trigger vulnerability analysis, View vulnerability warnings, Review suggested secure patches, Apply or reject generated fixes.
- The extension shall provide interactive notifications and messages regarding analysis status, detected vulnerabilities, and patch generation results.
- The system shall operate without requiring users to leave the VS Code environment.

4.6.2 Source Code Monitoring and Input Handling

- The system shall monitor source code files opened or modified within VS Code.
- The system shall support vulnerability analysis for multiple programming languages including C, C++, Rust, Java, Python.
- The extension shall automatically detect the programming language based on the file extension and code structure.
- The system shall analyze, Newly written code, Modified code sections, Complete project repositories.
- The system shall validate source files before analysis.
- The system shall handle, Unsupported file formats, Empty files, Corrupted source files, Invalid syntax errors.

4.6.3 Language Processing and LLVM IR Generation

- The system shall process only syntactically correct and compilable functions for LLVM Intermediate Representation (LLVM IR) generation.
- The system shall generate LLVM IR from supported programming languages using appropriate language-specific compiler frameworks:
 - Clang for C/C++.
 - rustc for Rust.
 - GraalVM for Java.
 - Nuitka for Python.
- The generated LLVM IR shall:
 - Provide a language-independent representation of source code.
 - Preserve the semantic structure and behavior of the program.
 - Represent control flow, data flow, function calls, memory operations, and instruction sequences.
 - Preserve sufficient function-level information for subsequent program analysis.
- The system shall maintain mapping between LLVM IR instructions and their corresponding source code locations.
- The system shall display syntax, compilation, or LLVM IR generation errors to the user.
- The system shall stop further analysis if syntax validation, compilation, or LLVM IR generation fails.

4.6.4 Semantic Vulnerability Context Extraction

- The system shall analyze generated LLVM IR to extract meaningful security-related information.

- The system shall generate Instruction-level Semantic Vulnerability Contexts (iSeVCs) containing:
 - Instruction sequences.
 - Data dependencies.
 - Control flow information.
 - Memory operations.
 - Function interactions
- The extracted semantic contexts shall be used as input for the LLM-based vulnerability detection module.
- The system shall reduce unnecessary code information before sending data to the LLM to improve efficiency and accuracy.

4.6.5 AI-Based Vulnerability Detection Using LLM

- The system shall integrate an LLM-based analysis module for intelligent vulnerability detection.
- The LLM shall analyze extracted semantic contexts and identify security vulnerabilities including:
 - Buffer overflow.
 - Null pointer dereference.
 - Memory leaks.
 - Use-after-free.
 - Integer overflow.
 - Unsafe pointer operations.
 - Improper input validation.
 - Logic-based security issues.
- The system shall classify detected vulnerabilities based on:

- Vulnerability category.
- Affected source code location.
- Severity level: Low, Medium, High, Critical.
- The system shall provide detailed vulnerability explanations.

4.6.6 Real-Time Vulnerability Visualization

- The system shall provide real-time vulnerability notifications inside the VS Code editor.
- Detected vulnerabilities shall be highlighted directly in the source code.
- The extension shall display:
 - Vulnerability type.
 - Severity level.
 - Explanation.
 - Suggested remediation.

4.6.7 Intelligent Secure Patch Generation

- After vulnerability detection, the system shall automatically invoke the LLM-based patch generation module.
- The system shall generate secure patch candidates based on:
 - Detected vulnerability type.
 - Program context.
- The system shall generate one or multiple patch suggestions when required.
- The user shall be able to:
 - Review generated patches.

- Compare different patch solutions.
 - Select the preferred patch.
 - Reject unwanted modifications.
- The system shall ensure that all code modifications require user approval before application.

4.6.8 Security Report Generation

- The system shall generate detailed vulnerability reports.
- The reports shall be exportable in formats such as PDF.
- The generated reports shall help developers understand the overall security status of their projects.

4.7 Non-Functional Requirements

Non-functional requirements specify the quality attributes and performance standards of a system rather than its specific functions. They define how the system should perform in terms of performance, reliability, scalability, security, usability, and maintainability, ensuring that the proposed system operates efficiently and meets user expectations.

4.7.1 Performance

The system performs real-time vulnerability detection with minimal impact on the performance of Visual Studio Code. Efficient semantic analysis using LLVM IR and optimized AI processing help reduce analysis time while maintaining high detection accuracy. The system delivers quick feedback, allowing developers to identify and fix vulnerabilities without significant delays.

4.7.2 Usability

The system provides a simple and user-friendly interface through the Visual Studio Code environment, allowing developers to use its features without extensive training. Vulnerability notifications, AI-generated explanations, secure patch suggestions, and security reports are presented in an intuitive manner. The seamless integration with Visual Studio Code enables developers to perform security analysis without interrupting their coding workflow, thereby improving productivity.

4.7.3 Reliability

The proposed system is designed to provide accurate and consistent vulnerability detection results using LLVM IR, iSeVCs, and Large Language Models. It verifies generated patches before they are applied to ensure that vulnerabilities are completely removed without introducing new defects. The system maintains stable performance during repeated analyses and generates reliable security reports that developers can trust.

4.7.4 Maintainability

The system follows a modular architecture, making it easier to update, maintain, and extend individual components. AI models, LLVM analysis modules, vulnerability databases, and security rules can be upgraded independently without affecting the overall system. This modular design reduces maintenance effort and allows future enhancements to be integrated efficiently.

4.7.5 Explainability

The proposed system provides Explainable Artificial Intelligence (XAI) to ensure that the results generated by the AI are clear and understandable to developers. Instead of only identifying vulnerabilities, the system explains the root cause, severity, potential security impact, and recommended secure coding practices for each detected issue. These explanations help developers

understand why a vulnerability occurs and how it can be effectively resolved. By providing transparent and meaningful insights, the system improves developer confidence, supports secure coding practices, and enables informed decision-making during software development.

4.8 Feasibility Study

A feasibility study is the process of evaluating whether a proposed system can be developed and implemented successfully. It analyzes different factors such as technical capability, economic viability, operational efficiency, and project schedule to determine whether the system is practical and beneficial.

4.8.1 Technical Feasibility

Technical feasibility evaluates whether the proposed system can be developed using the available technologies, tools, and technical resources. FlawFix SecureAssist is technically feasible because it is built using modern and widely adopted technologies such as Visual Studio Code Extension API, LLVM Framework, Python, Large Language Models (LLMs), and Explainable Artificial Intelligence (XAI). The system uses LLVM Intermediate Representation (LLVM IR) to perform language-independent semantic analysis and generate Intermediate Semantic Vulnerability Contexts (iSeVCs) for accurate vulnerability detection. LLMs are integrated to analyze vulnerabilities, generate explanations, create secure patches, verify applied fixes, and recommend code optimization. The modular architecture allows individual components to be developed, tested, and updated independently, making the system reliable, scalable, and suitable for future enhancements. Therefore, the required technologies and development environment are readily available, making the proposed system technically feasible.

4.8.2 Operational Feasibility

Operational feasibility evaluates whether the proposed system can be effectively used by its intended users in a real-world environment. FlawFix

SecureAssist is operationally feasible because it integrates directly into the Visual Studio Code development environment, allowing developers to receive real-time vulnerability detection, AI-generated explanations, secure patch suggestions, patch verification, and code optimization without leaving the IDE. The user-friendly interface minimizes the learning curve and enables developers to adopt the system with minimal training. The system automates several security-related tasks, reducing manual effort while improving productivity and software quality. Comprehensive security reports and AI-assisted recommendations help developers make informed decisions during software development. As a result, the proposed system provides an efficient, practical, and reliable solution for secure software development and can be easily adopted in academic, research, and industrial environments

4.8.3 *Economical Feasibility*

Economic feasibility determines whether the proposed system can be developed and maintained within a reasonable cost. FlawFix SecureAssist is economically feasible because it primarily uses open-source software and development tools, including Visual Studio Code, LLVM Framework, Python, Git, and various AI libraries. These technologies significantly reduce software licensing costs while providing powerful development capabilities. The hardware requirements are moderate and can be satisfied using a standard computer with an Intel Core i5 or AMD Ryzen 5 processor, 8 GB RAM, and SSD storage. Furthermore, the system reduces the manual effort required for vulnerability detection, secure patch generation, and verification, thereby saving development time and reducing maintenance costs. Although cloud-based LLM services may involve operational costs, open-source or locally deployed models can also be used, making the overall implementation cost-effective.

CHAPTER 5

SYSTEM DESIGN

The design of FlawFix SecureAssist is presented in this chapter, focusing on the organization and interaction of the major system components involved in real-time vulnerability detection and secure patch generation. The system follows a modular pipeline in which source code is processed through LLVM IR generation, iSeVC feature extraction, LLM-based vulnerability detection, Explainable AI, secure patch generation, patch validation, verification and security report generation.

5.1 System Architecture

As depicted in Figure 5.1, the architecture of FlawFix SecureAssist represents an AI-driven pipeline for vulnerability detection and secure patch generation. The process begins when the user writes or uploads source code through Visual Studio Code. The FlawFix SecureAssist extension monitors the code and supports C, C++, Rust, Java, and Python.

The source code is processed by the LLVM IR Analysis Engine, which generates LLVM Intermediate Representation using suitable language-specific tools. The generated LLVM IR is then processed by the iSeVC Feature Extraction module to obtain semantic information required for vulnerability analysis.

The extracted information is provided to the LLM-Based Vulnerability Detection module. If a vulnerability is detected, the Explainable AI module provides information about the vulnerability, root cause, and severity. The Secure Patch Generator then generates multiple secure patch suggestions, which are displayed through the Patch Preview and Validation module for

user selection.

The approved patch is applied to the source code. If no vulnerability is detected, the system generates the security report and provides the secure source code. The architecture integrates vulnerability detection, explanation, secure patch generation, validation, and reporting within the Visual Studio Code environment.

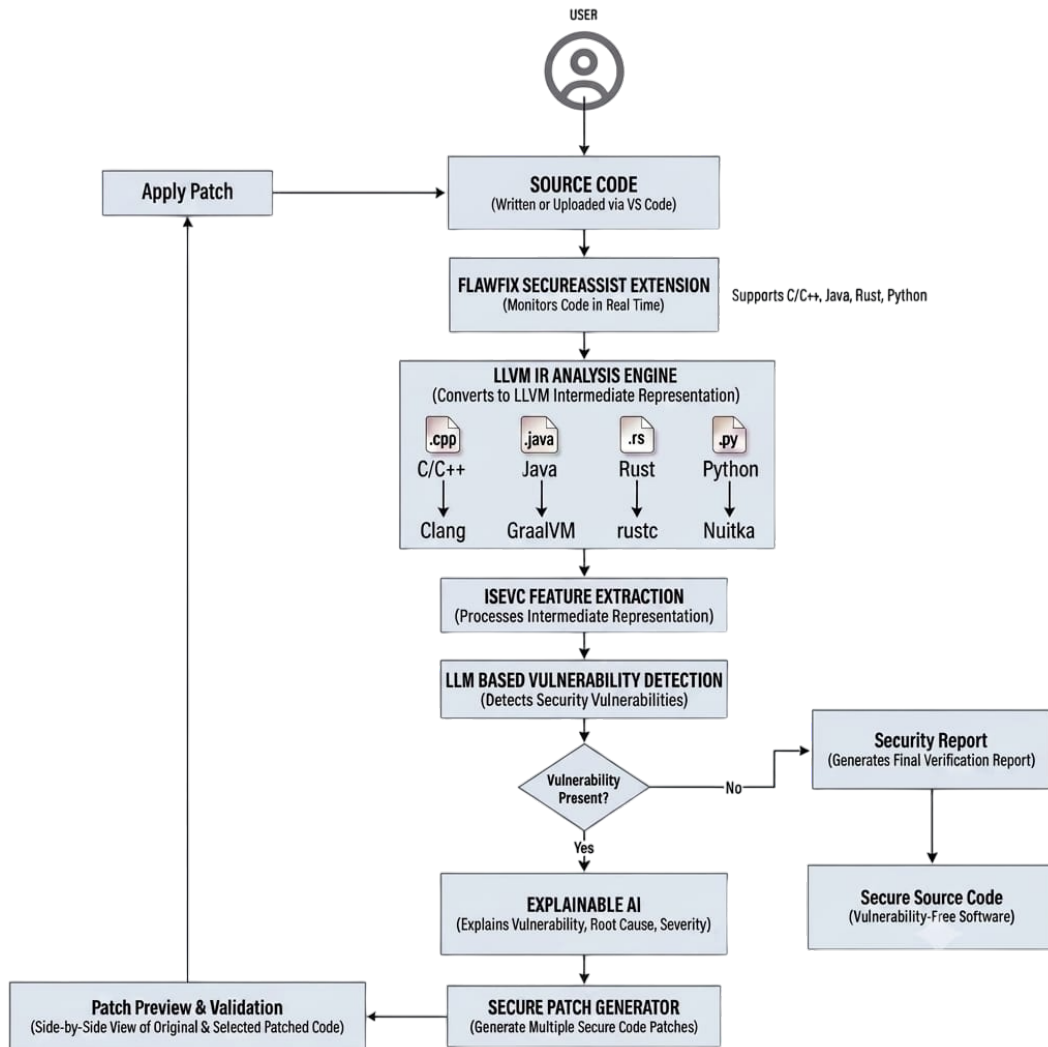


Fig. 5.1: System Architecture of FlawFix SecureAssist.

5.2 Data Flow Diagram (DFD)

Figures 5.2 and 5.3 illustrate the Data Flow Diagrams of the proposed system, showing the movement of source code, patch information, and

security-related outputs throughout the vulnerability detection and secure patch generation process.

5.2.1 DFD Level 0

Figure 5.2 demonstrates the Level 0 Data Flow Diagram (DFD) of the FlawFix SecureAssist System. The user provides source code and patch approval to the system. The system analyzes the source code for vulnerabilities, generates secure patch suggestions, and produces a security report. The resulting secure source code, secure patch suggestions, and security report are returned to the user.

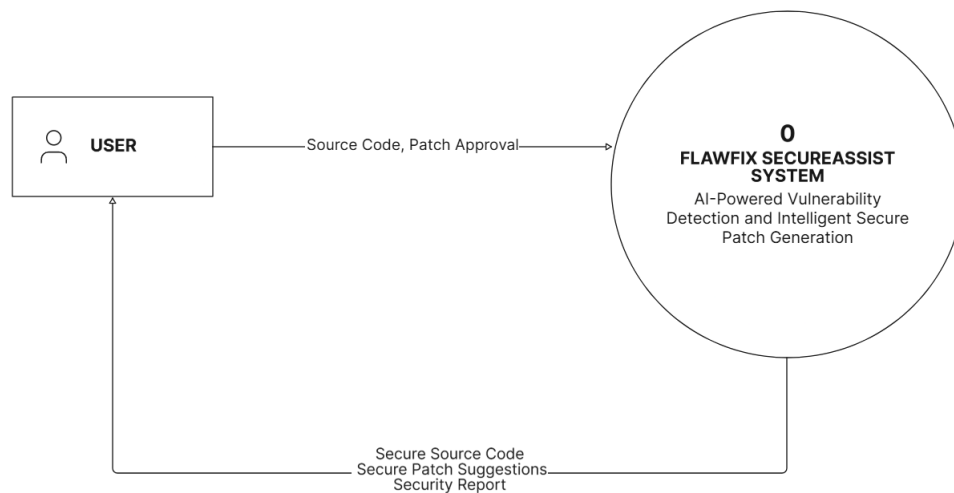


Fig. 5.2: Data Flow Diagram Level 0.

5.2.2 DFD Level 1

Figure 5.3 presents the internal processing of FlawFix SecureAssist as a Level 1 DFD. The process begins when the user provides source code to the FlawFix SecureAssist Extension. The extension checks the source code and forwards syntactically correct code to the LLVM IR Generator. The generated LLVM IR is then passed to the iSeVC Feature Extractor to obtain relevant semantic features.

The extracted features are provided to the LLM-Based Vulnerability Detection module, which identifies potential vulnerabilities. The detected vulnerabilities are passed to the Explainable AI module to generate information such as the root cause, severity, and explanation. The vulnerability information is then provided to the Secure Patch Generation module, which generates multiple secure patch alternatives.

The generated patches are passed to the Patch Preview and Validation module, where the user can review the proposed changes and provide patch approval. The approved patch is then applied to the source code through the Apply Patch module. The updated secure source code and security report are returned to the user. If LLVM IR generation fails, the corresponding error is returned to the user.

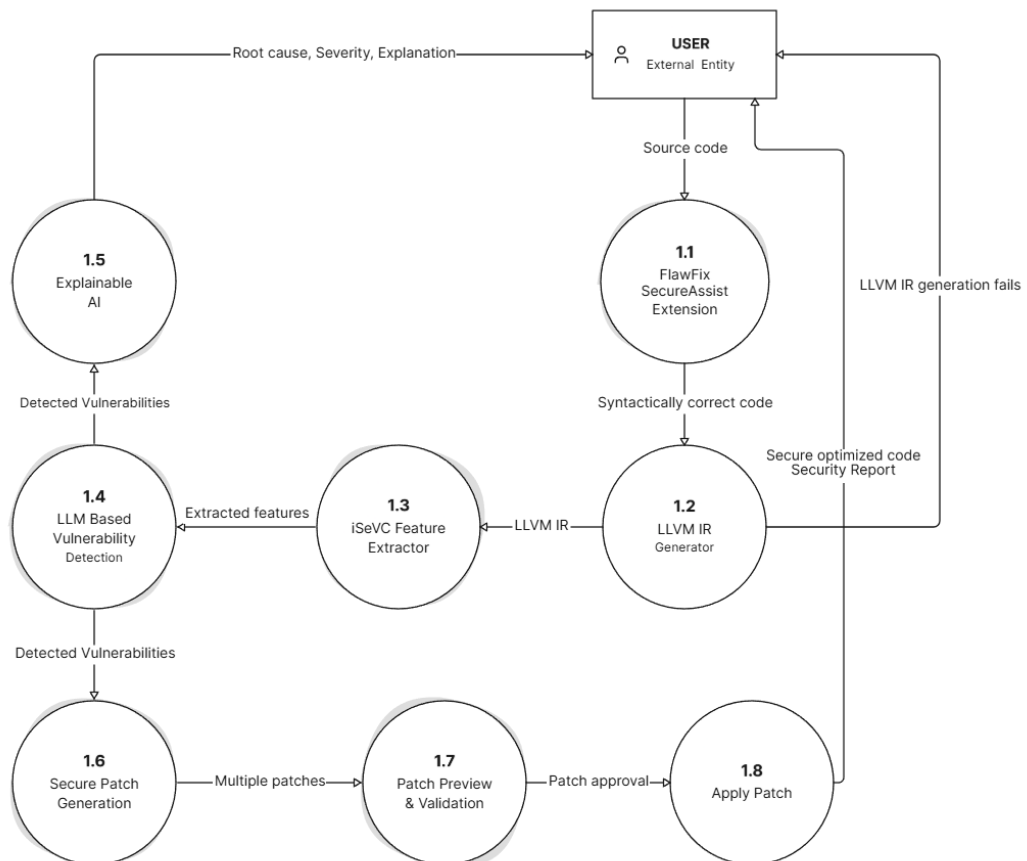


Fig. 5.3: Data Flow Diagram Level 1.

5.3 List of Modules

FlawFix SecureAssist is composed of the following modules:

- FlawFix SecureAssist Extension
- LLVM IR Generator
- iSeVC Feature Extractor
- LLM-Based Vulnerability Detection
- Explainable AI
- Secure Patch Generation
- Patch Preview and Validation
- Apply Patch

5.3.1 FlawFix SecureAssist Extension

This module provides the interface between the user and the security analysis system. It monitors source code written or uploaded in Visual Studio Code and forwards the source code for further analysis. It also receives analysis results, secure code, and security reports for display to the user.

5.3.2 LLVM IR Generator

This module converts syntactically correct source code into LLVM Intermediate Representation (LLVM IR) using suitable language-specific tools. The generated LLVM IR provides a common representation for further security analysis. If IR generation fails, the error is returned to the user and further analysis is stopped.

5.3.3 iSeVC Feature Extractor

This module processes the generated LLVM IR and extracts Intermediate Semantic Vulnerability Contexts (iSeVCs). These contexts contain relevant

semantic information required for identifying and analyzing potential software vulnerabilities.

5.3.4 LLM-Based Vulnerability Detection

This module uses a Large Language Model to analyze the extracted semantic information and detect potential vulnerabilities. It identifies vulnerable code and provides relevant vulnerability information for further explanation and patch generation.

5.3.5 Explainable AI

This module provides understandable explanations for detected vulnerabilities. It identifies the root cause and severity of the vulnerability and presents the security issue in a form that can be easily understood by the developer.

5.3.6 Secure Patch Generation

This module generates multiple secure patch alternatives for the detected vulnerability. The generated patches aim to remove the vulnerability while preserving the intended functionality of the source code.

5.3.7 Patch Preview and Validation

This module presents the generated patches for user review and validation. It allows the developer to compare the original and proposed code and provide approval for the selected patch.

5.3.8 Apply Patch

This module applies the approved patch to the source code and produces the updated secure code. The resulting secure code and security report are provided to the user.

In this chapter, the overall system design of FlawFix SecureAssist is discussed, including the system architecture, major modules, and complete

data flow. The modular design enables source code to be systematically processed from LLVM IR generation and semantic analysis to vulnerability detection, explanation, secure patch generation, validation, and patch application.

BIBLIOGRAPHY

- [1] Akhila Mangalam P S . Raseek C. Flawfix: Language-agnostic vulnerability detection and explanation using llvm ir and large language models, government engineering college thrissur, 2026.
- [2] Y. Liu et al. Software vulnerability detection using large language models. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 2023.
- [3] S. Li et al. Research on the llm-driven vulnerability detection system using lprotector. In *Proceedings of the IEEE International Conference on Data Science and Computing Applications (ICDSCA)*, 2024.
- [4] X. Chen et al. Dependable code repair with llms: Ai-driven vulnerability detection and automated patching. In *Proceedings of the IEEE Pacific Rim International Symposium on Dependable Computing (PRDC)*, 2025.
- [5] J. Smith et al. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 34(2), 2025.